

Fase 1 — Definición y entorno reproducible

Curso: MCDI500

Grupo: 4

Integrantes: Felipe Díaz Toro, Pablo Ríos Passteni, Guillermo Subiabre Chacón, Ninoska Yévenes Hernández

Contexto y problema

Las enfermedades cardiovasculares (ECV) son la principal causa de muerte a nivel mundial: en 2023 provocaron 19,2 millones de fallecimientos —una de cada tres muertes globales—, cifra que casi se ha duplicado desde los 13,1 millones registrados en 1990 (Global Burden of Disease, 2023). A esto se suma una carga considerable de discapacidad (437 millones de años de vida ajustados por discapacidad) y una prevalencia de hipertensión arterial que afecta a 1.280 millones de adultos entre 30 y 79 años en el mundo, de los cuales solo uno de cada cinco mantiene su condición controlada (OMS, 2023).

Chile no escapa a esta tendencia. Las enfermedades cardiovasculares representan una de las principales causas de morbilidad en el país, y sus indicadores de riesgo (hipertensión, diabetes, dislipidemia, obesidad y sedentarismo) presentan distribuciones heterogéneas según el perfil sociodemográfico de la población. Por ello, este proyecto busca investigar cómo se relacionan diversos factores sociodemográficos, como la edad, el sexo, el nivel socioeconómico y el nivel educacional, con factores de riesgo cardiovascular en la población chilena. Para ello, el presente estudio analiza datos de la **Encuesta Nacional de Salud (ENS) 2016–2017**, desarrollada por el Ministerio de Salud de Chile (MINSAL), con el propósito de identificar posibles asociaciones entre las características sociodemográficas de la población y la presencia de factores de riesgo cardiovascular.

Relevancia técnica e informática: La complejidad y volumen de la encuesta exige un enfoque computacional reproducible para procesar códigos de no respuesta, ponderadores muestrales y variables categóricas complejas, lo que imposibilita un análisis manual directo.

Caracterización del Conjunto de Datos y Roles Analíticos

- **Fuente:** Encuesta Nacional de Salud (ENS 2016–2017, MINSAL).
- **Unidad de observación:** Persona encuestada residente en Chile.
- **Dimensiones:** Subconjunto de 5520 de un total **6.233 registros** (filas) y **16 variables** (columnas).

Clasificación de variables por rol analítico:

1. **Identificadores y diseño muestral:** IdEncuesta , FechaInicioF1 , Estrato , Conglomerado , y factores de expansión (Fexp_F1F2p_Corr).
2. **Exposiciones / Sociodemográficas:** Edad (años), Sexo (masculino o femenino), Zona de residencia (Rural o Urbano), Años de estudio (anos_estudio_MINSAL_1) e Ingreso del hogar (as27 , as28).
3. **Indicadores y diagnósticos clínicos (Resultados):**
 - Diagnósticos autorreportados de hipertensión (HTA), diabetes (di3) y colesterol alto (dis2).
 - Mediciones físicas y estilo de vida: Índice de Masa Corporal (IMC) y Actividad física (GPAQ).

Pregunta de investigación

¿Cómo se relacionan las condiciones sociodemográficas (edad, el sexo, el nivel educacional, el ingreso del hogar y la zona de residencia) con la presencia de hipertensión arterial, diabetes, colesterol alto, índice de masa corporal y el nivel de actividad física en los participantes de la ENS 2016–2017?

Objetivo general

Declarar y caracterizar la problemática de salud pública sobre el riesgo cardiovascular en Chile a partir de la Encuesta Nacional de Salud (ENS 2016–2017), mediante la implementación de un entorno computacional reproducible y trazable en Python, Git y GitHub que sostenga el ciclo completo de procesamiento y análisis.

Objetivos específicos

1. Definir la problemática y asegurar trazabilidad: Formular el problema de investigación, las preguntas centrales y los criterios de éxito del proyecto, garantizando la correspondencia entre el mapa conceptual, la estructura del repositorio y el cuaderno de la Fase 1.
2. Construir y verificar el entorno reproducible: Implementar la arquitectura del repositorio versionado en Git/GitHub (carpetas por fase, README.md, .gitignore y requirements.txt) y comprobar mediante código que el kernel de ejecución corresponde al entorno virtual del proyecto (.venv) y no al sistema.
3. Documentar procedencia y diagnosticar calidad de datos: Evaluar la fuente de la ENS 2016–2017 y seleccionar las variables clave identificando en su estructura inicial la presencia de inconsistencias, valores faltantes y códigos especiales de no respuesta según sus libros de códigos.

4. Implementar el pipeline modular de preprocesamiento (Fase 2): Construir e integrar funciones reutilizables para la limpieza, recodificación y transformación de variables sociodemográficas y clínicas, justificando técnicamente el tratamiento de faltantes y casos límite.
5. Analizar y comunicar estadísticamente las relaciones (Fases 3 y 4): Evaluar las asociaciones entre el perfil sociodemográfico (edad, sexo, educación, ingreso y zona) y los indicadores de riesgo cardiovascular considerando el diseño muestral y sus limitaciones, comunicando los hallazgos de manera rigurosa y reproducible

Alcance de este avance

La Fase 1 documenta el problema, los objetivos y la configuración inicial del entorno.

- **En este notebook (Fase 1) no se realiza limpieza, imputación ni transformación de variables**, tareas reservadas estrictamente para la Fase 2

La Fase 2 abordará la obtención, exploración, limpieza, transformación y validación del subconjunto seleccionado.

- El análisis de asociaciones se desarrollará en las fases posteriores.
- El estudio no permitirá atribuir causalidad ni estimará un puntaje global de riesgo cardiovascular.

Supuestos clave: Se asume que las respuestas registradas en el instrumento reflejan la condición del encuestado y que los códigos de no respuesta serán tratados mediante reglas explícitas de imputación o filtrado documentadas en el diccionario de datos

```
In [1]: import sys
from pathlib import Path
import numpy as np           # presente para verificar el entorno del pr
import pandas as pd         # tablas de documentación de la fase

# Reproducibilidad: la misma semilla que usa el cuaderno de la Fase 2.
SEMILLA = 2026
np.random.seed(SEMILLA)

#print("Cuaderno de la Fase 1 .", date.today().isoformat())
print(sys.executable)
```

/Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/.env/bin/python

1. Definición del problema

Esta sección actúa como una ficha de configuración global dentro del código.

Centralización de datos: Agrupa la información técnica y administrativa del proyecto en un único lugar (diccionario) para no repetirla en el resto del script.

Trazabilidad y reproducibilidad: Registra parámetros clave (rutas de archivo, conteo de datos y semillas aleatorias) para asegurar que el análisis sea consistente y fácil de replicar.

Verificación rápida: Imprime en pantalla todas las variables configuradas para confirmar que se cargaron correctamente antes de ejecutar el análisis.

```
In [2]: # Metadatos reutilizables del proyecto – fuente unica de estos datos.
# La narrativa completa (problematica, objetivos, alcance) vive en
# las celdas Markdown de arriba; este diccionario NO la repite.
PROYECTO = {
    "titulo": "Factores sociodemograficos y riesgo cardiovascular (ENS 2016-2017)",
    "grupo": "Grupo 4",
    "asignatura": "MCDI500 – Programacion para la Ciencia de Datos",
    "integrantes": [
        "Felipe Diaz Toro",
        "Pablo Rios Passteni",
        "Guillermo Subiabre Chacon",
        "Ninoska Yevenes Hernandez",
    ],
    "fuente_datos": "Encuesta Nacional de Salud (ENS) 2016–2017, MINSAL",
    "archivo_base": "data/raw/ens2016.xlsx",
    "ponderador": "Fexp_F1F2p_Corr",
    "n_total_base": 6233,
    "n_elegible_f1f2": 5520,
    "semilla": SEMILLA,
}

print("Metadatos del proyecto cargados:")
for clave, valor in PROYECTO.items():
    print(f" {clave}: {valor}")
```

```
Metadatos del proyecto cargados:
  titulo: Factores sociodemograficos y riesgo cardiovascular (ENS 2016–2017)
  grupo: Grupo 4
  asignatura: MCDI500 – Programacion para la Ciencia de Datos
  integrantes: ['Felipe Diaz Toro', 'Pablo Rios Passteni', 'Guillermo Subiabre Chacon', 'Ninoska Yevenes Hernandez']
  fuente_datos: Encuesta Nacional de Salud (ENS) 2016–2017, MINSAL
  archivo_base: data/raw/ens2016.xlsx
  ponderador: Fexp_F1F2p_Corr
  n_total_base: 6233
  n_elegible_f1f2: 5520
  semilla: 2026
```

Esta sección actúa como una validación de entorno y reproducibilidad antes de ejecutar el análisis.

Inspección de dependencias: Identifica e imprime las versiones exactas de Python y las librerías clave (NumPy, pandas, matplotlib, sklearn) para evitar inconsistencias o fallos por compatibilidad.

Control de entorno virtual: Valida mediante una aserción (assert) que el cuaderno se esté ejecutando obligatoriamente dentro del entorno virtual asignado (.venv).

Trazabilidad de contexto: Registra la ruta del ejecutable de Python y el directorio de trabajo activo para garantizar la correcta localización de archivos y módulos locales.

```
In [3]: # Verificacion del entorno reproducible
print(f"Python: {sys.version.split()[0]}")
print(f"NumPy: {np.__version__}")
print(f"pandas: {pd.__version__}")
for lib in ["matplotlib", "sklearn"]:
    try:
        print(f" {lib:12}", __import__(lib).__version__)
    except ImportError:
        print(f" {lib:12} NO DISPONIBLE en este kernel")
print(f"Ejecutable: {sys.executable}")
print("Carpeta de trabajo:", Path.cwd())

assert ".venv" in sys.executable, "El notebook no esta usando el entorno vir
print("\nEntorno verificado correctamente.")
```

Python: 3.13.7

NumPy: 2.5.3

pandas: 3.0.5

matplotlib 3.11.1

sklearn 1.9.0

Ejecutable: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/.venv/bin/python

Carpeta de trabajo: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/F1/notebooks

Entorno verificado correctamente.

Esta sección es una función de validación de metadatos.

Chequeo de claves obligatorias: Compara las claves del diccionario del proyecto contra un conjunto predefinido de campos requeridos (como título, grupo, integrantes, etc.).

Detección de faltantes: Utiliza la resta de conjuntos (campos_requeridos - proyecto_dict.keys()) para identificar al instante qué datos faltan.

Retorno booleano y alerta: Devuelve True si la estructura está completa o False junto con un mensaje en consola indicando los campos pendientes, asegurando que no se ejecuten pasos posteriores sin la información necesaria.

```
In [4]: def verificar_estructura_proyecto(proyecto_dict: dict) -> bool:
        """
        Comprueba que el diccionario de metadatos tenga los campos
        minimos necesarios para generar el README y la tabla de
        vinculacion con el mapa conceptual.
        """
        campos_requeridos = {
            "titulo", "grupo", "integrantes", "fuente_datos",
```

```

        "archivo_base", "ponderador", "n_elegible_f1f2",
    }
    faltantes = campos_requeridos - proyecto_dict.keys()
    if faltantes:
        print(f"Faltan campos: {faltantes}")
        return False
    print("Metadatos completos y listos para reutilizar.")
    return True

verificar_estructura_proyecto(PROYECTO)

```

Metadatos completos y listos para reutilizar.

Out[4]: True

2. Estructura del proyecto y reconocimiento del conjunto de datos

Esta sección realiza la localización dinámica de rutas y la carga comparativa de datasets (base original vs. subconjunto de trabajo).

Detección de raíz del proyecto: Utiliza encontrar_raiz_proyecto para ubicar automáticamente la carpeta principal mediante el marcador .git, asegurando que el código sea portátil.

Carga doble de datos: Lee tanto la base de datos de origen completa sin modificar (data/raw/ens2016.xlsx) como el subconjunto de variables ya procesado (data/processed/ens_variables_f1f2.xlsx).

Inspección y validación: Imprime en consola las dimensiones de ambas bases (permitiendo comparar volumen original vs. filtrado), además de los tipos de datos y una vista previa (.head()) de la base de trabajo.

```

In [5]: def encontrar_raiz_proyecto(marcador=".git") -> Path:
        """Sube por las carpetas padre hasta encontrar la raíz del repositorio."""
        actual = Path.cwd()
        for carpeta in [actual, *actual.parents]:
            if (carpeta / marcador).exists():
                return carpeta
        raise FileNotFoundError(f"No se encontro '{marcador}' en ningun directorio")

RAIZ = encontrar_raiz_proyecto()
print("Raiz del proyecto detectada en:", RAIZ)

# 1) Reconocer la base origen (sin modificar, tal como llega de MINSAL)
ARCHIVO_ORIGEN = RAIZ / "data" / "raw" / "ens2016.xlsx"
print("Cargando la base origen (puede tardar unos segundos por su tamaño)...")
df_origen = pd.read_excel(ARCHIVO_ORIGEN)
print("Base origen reconocida:", ARCHIVO_ORIGEN)
print("Dimensiones de la base origen:", df_origen.shape)

# 2) Reconocer el subconjunto de trabajo (las 16 variables ya seleccionadas)

```

```

ARCHIVO_SELECCIONADO = RAIZ / "data" / "processed" / "ens_variables_f1f2.xls
df_seleccion = pd.read_excel(ARCHIVO_SELECCIONADO)

print("\nArchivo de trabajo reconocido:", ARCHIVO_SELECCIONADO)
print("Dimensiones:", df_seleccion.shape)
print("\nTipos de datos:")
print(df_seleccion.dtypes)

df_seleccion.head()

```

Raiz del proyecto detectada en: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500

Cargando la base origen (puede tardar unos segundos por su tamaño)...

Base origen reconocida: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/raw/ens2016.xlsx

Dimensiones de la base origen: (6233, 1173)

Archivo de trabajo reconocido: /Users/nyevenes/DevProjects/MagisterIA/proyecto-grupo4-mcdi500/data/processed/ens_variables_f1f2.xlsx

Dimensiones: (5520, 16)

Tipos de datos:

```

IdEncuesta                int64
FechaInicioF1             datetime64[us]
Edad                      int64
Sexo                      int64
Zona                      int64
HTA                       float64
di3                       int64
dis2                      int64
IMC                       float64
anos_estudio_MINSAL_1     float64
GPAQ                     float64
as27                     float64
as28                      int64
Fexp_F1F2p_Corr          float64
Conglomerado             int64
Estrato                  int64
dtype: object

```

Out[5]:

	IdEncuesta	FechaInicioF1	Edad	Sexo	Zona	HTA	di3	dis2	IMC	anos_es
--	------------	---------------	------	------	------	-----	-----	------	-----	---------

0	20008	2016-11-27 15:10:40	25	2	1	0.0	2	3	23.198208	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

1	20011	2016-11-26 15:52:49	20	2	1	0.0	2	3	26.203983	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

2	20012	2016-11-20 13:16:39	85	2	1	1.0	2	2	23.732346	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

3	20017	2016-12-02 15:25:24	30	2	1	0.0	2	3	25.015243	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

4	20018	2016-11-26 16:02:05	53	2	1	0.0	2	3	32.290249	
---	-------	------------------------	----	---	---	-----	---	---	-----------	--

Esta sección realiza una auditoría de integridad del repositorio.

Definición de arquitectura: Define en una lista (ESTRUCTURA_ESPERADA) los archivos y carpetas obligatorios que deben existir en el proyecto (cuadernos, datos, código fuente, documentación y configuración de Git).

Verificación de existencia: Recorre la lista y comprueba la existencia de cada elemento de forma relativa a la ruta raíz del repositorio.

Validación e informe: Muestra un listado detallado si detecta carpetas o archivos faltantes (retornando False), o confirma que el proyecto cumple con el estándar de organización esperado (retornando True).

```
In [6]: ESTRUCTURA_ESPERADA = [
        "F1/notebooks",
        "F2/notebooks",
        "data/raw",
        "data/processed",
        "docs",
        "src",
        "tests",
        "README.MD",
        "requirements.txt",
        ".gitattributes",
        ".gitignore",
    ]

def verificar_estructura_repositorio(raiz: Path, elementos_esperados: list)
    """Comprueba que existan las carpetas y archivos clave del repositorio."""
    faltantes = [e for e in elementos_esperados if not (raiz / e).exists()]
    if faltantes:
        print("Faltan estos elementos en la estructura del repositorio:")
        for f in faltantes:
            print(f" - {f}")
        return False
    print("Estructura del repositorio verificada: todos los elementos esperados existen.")
    return True

verificar_estructura_repositorio(RAIZ, ESTRUCTURA_ESPERADA)
```

Estructura del repositorio verificada: todos los elementos esperados existen.

Out[6]: True